Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 8

Aim

Design and train a Convolutional Neural Network (CNN) on the MNIST image dataset for multi-class digit classification, and evaluate model performance with and without hyperparameter tuning.

---

1. Dataset Source

- Dataset: MNIST (Modified National Institute of Standards and Technology)
- Built into Keras: keras.datasets.mnist — no external download required
- Original source: Yann LeCun, Corinna Cortes, Christopher J.C. Burges
- Website: http://yann.lecun.com/exdb/mnist/

---

2. Dataset Description

The MNIST dataset is the most widely used benchmark for image classification algorithms. It consists of handwritten digit images collected from US Census Bureau employees and high school students. Each image is a grayscale 28×28 pixel scan of a single handwritten digit (0–9).

Key properties:

- Total size: 70,000 images — 60,000 training, 10,000 test
- Image dimensions: 28 × 28 pixels, single grayscale channel (shape: 28×28×1)
- Target variable: Digit class (0–9), 10 classes
- Class distribution: ~6,000 training examples per class — approximately balanced across all digits
- Preprocessing applied: Pixel values normalized to [0, 1] by dividing by 255; reshaped to (N, 28, 28, 1) to add the channel dimension required by Conv2D

---

3. Mathematical Formulation of the Algorithm

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed for grid-structured data such as images. Unlike a standard ANN that applies

fully connected transformations from the input, a CNN applies local, weight-shared filters to extract spatial features before passing them to a classifier head.

3.1 Discrete 2D Convolution

For an input feature map I and a learned filter K of size k×k, the output feature map S at position (i, j) is:

$$S(i, j) = (I * K)(i, j) = \Sigma \square \Sigma \square \; I(i+m, j+n) \cdot K(m, n) + b$$

where b is a bias term. Each filter slides over the entire input, computing dot products to detect a specific local pattern (edge, corner, curve). Multiple filters produce multiple feature maps (channels) per convolutional layer.

3.2 ReLU Activation

Applied element-wise to each feature map after convolution to introduce non-linearity and enable the network to learn complex decision boundaries:

$$ReLU(z) = max(0, z)$$

3.3 Max Pooling

A down-sampling operation that reduces spatial dimensions by retaining only the maximum value in each non-overlapping pooling window of size p×p:

$$Pool(i, j) = max^{-} \square^{-n} \{ S(i \cdot p + m, \; j \cdot p + n) \}$$

A 2×2 max pool halves both height and width of the feature map, reducing parameters and providing local translation invariance.

3.4 Flatten and Fully Connected Layers

After all convolutional and pooling layers, the multi-dimensional feature maps are flattened into a 1D vector and passed through Dense layers:

$$a^l = ReLU(W^l \cdot a^{l-1} + b^l)$$

3.5 Softmax Output (Multi-Class Classification)

The final Dense(10) layer uses Softmax to convert raw logits into a probability distribution over all 10 digit classes:

$$\sigma(z)_i = e_{r^i} / \Sigma_j \; e_{rj}$$

The predicted class is the digit with the highest probability: $\hat{y} = \text{argmax}_i \; \sigma(z)_i$.

3.6 Categorical Cross-Entropy Loss

For m training samples over C=10 classes with true labels y and predicted probabilities $\hat{p}$:

$$J = -(1/m) \; \Sigma_i \; \Sigma c \; y_i c \cdot log(\hat{p}_i c)$$

In this experiment, sparse_categorical_crossentropy is used, which accepts integer class labels directly (no one-hot encoding needed).

3.7 Backpropagation and Adam Optimizer

Gradients are propagated backwards through all layers including the convolutional filters via the chain rule. Adam updates each parameter adaptively:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1\times10^{-7}$. Learning rate $\alpha = 0.001$ in both baseline and best tuned model.

---

4. Algorithm Limitations

1. Computationally Intensive

CNNs involve large numbers of matrix multiplications across filter weights during forward and backward passes. Training on large datasets demands GPU/TPU acceleration. Even on MNIST with 60,000 images, each epoch processes millions of convolution operations across all filter-position combinations.

2. Designed Specifically for Grid-Structured Data

The spatial locality assumption embedded in convolution makes CNNs ideal for images but unsuitable for tabular or sequential data without forcing data into a grid structure. Applying a CNN to non-image data would destroy meaningful feature relationships.

3. Large Amount of Labeled Data Required

CNNs have many learnable parameters (filter weights across multiple convolutional layers). Without sufficient labeled examples, the model cannot learn generalizable filters and will overfit to the training set. Transfer learning from pre-trained networks (VGG, ResNet) is a common mitigation for small datasets.

4. Sensitivity to Hyperparameters

Performance depends heavily on the number of convolutional layers, filter counts, filter sizes, pooling strategy, dense head size, dropout rate, and learning rate. Poor choices lead to slow convergence, vanishing gradients, or underfitting. Systematic tuning is essential for pushing beyond baseline performance.

5. Limited Invariance Without Data Augmentation

Standard CNNs achieve local translation invariance via pooling, but are not inherently robust to rotation, scale changes, or perspective distortions. These require explicit data augmentation (random rotations, flips, crops) during training. No augmentation was applied in this experiment as MNIST digits are already standardized.

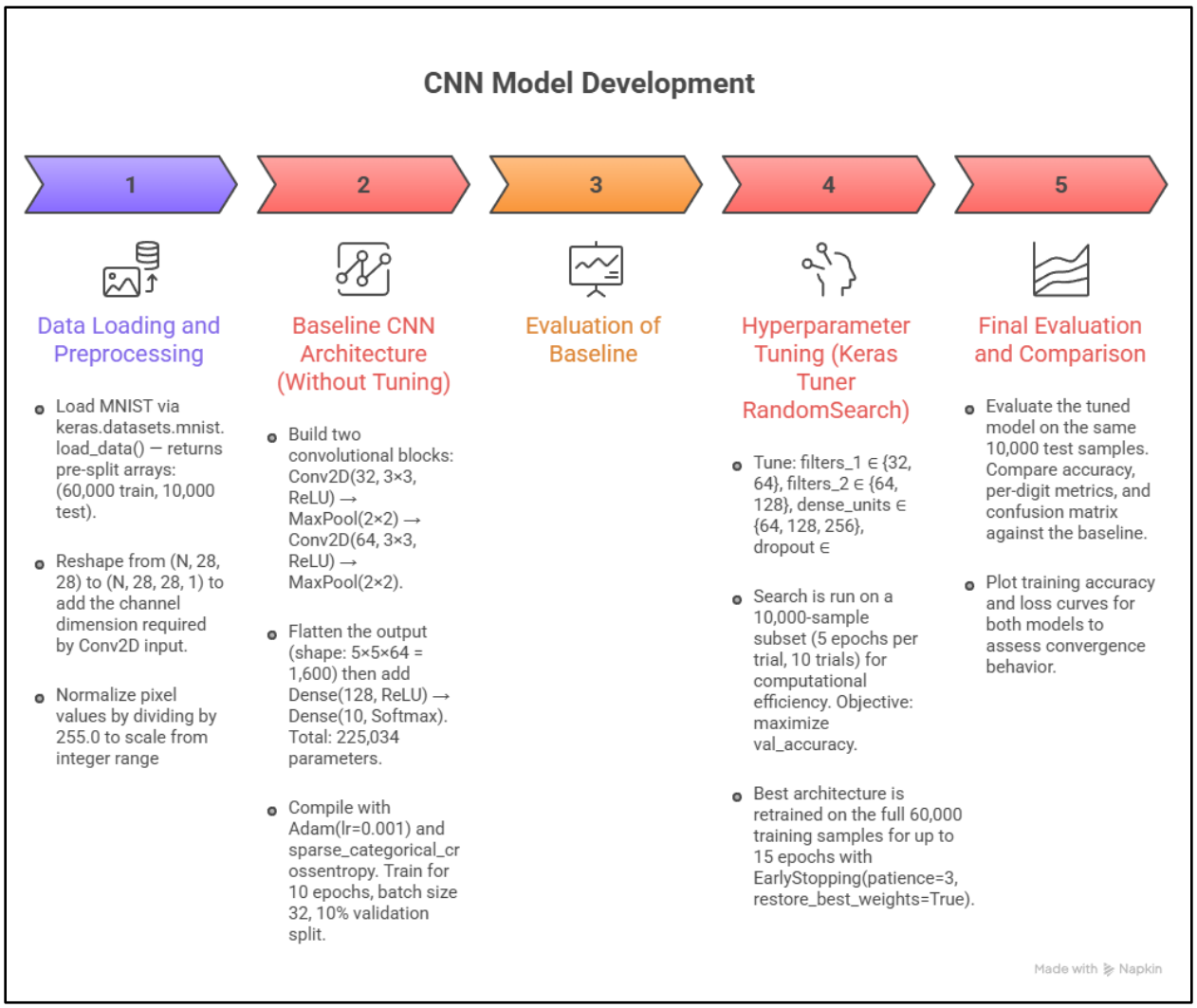
6. Black-Box Interpretability

The hierarchical feature extraction process — from edges and curves in early layers to complex digit strokes in deeper layers — is not human-interpretable without dedicated visualization techniques such as Grad-CAM, feature map visualization, or activation maximization.

---

5. Methodology / Workflow

1. **Data Loading and Preprocessing**
   - Load MNIST via keras.datasets.mnist.load_data() — returns pre-split arrays: (60,000 train, 10,000 test).
   - Reshape from (N, 28, 28) to (N, 28, 28, 1) to add the channel dimension required by Conv2D input.
   - Normalize pixel values by dividing by 255.0 to scale from integer range [0, 255] to float range [0.0, 1.0].
2. **Baseline CNN Architecture (Without Tuning)**
   - Build two convolutional blocks: Conv2D(32, 3×3, ReLU) → MaxPool(2×2) → Conv2D(64, 3×3, ReLU) → MaxPool(2×2).
   - Flatten the output (shape: 5×5×64 = 1,600) then add Dense(128, ReLU) → Dense(10, Softmax). Total: 225,034 parameters.
   - Compile with Adam(lr=0.001) and sparse_categorical_crossentropy. Train for 10 epochs, batch size 32, 10% validation split.
3. **Evaluation of Baseline**
   - Evaluate on 10,000 test images. Record test accuracy, per-digit classification report (precision, recall, F1, support), confusion matrix, and training curves.
4. **Hyperparameter Tuning (Keras Tuner RandomSearch)**
   - Tune: filters_1 ∈ {32, 64}, filters_2 ∈ {64, 128}, dense_units ∈ {64, 128, 256}, dropout ∈ [0.2, 0.5], learning_rate ∈ {1e–2, 1e–3, 1e–4}.
   - Search is run on a 10,000-sample subset (5 epochs per trial, 10 trials) for computational efficiency. Objective: maximize val_accuracy.
   - Best architecture is retrained on the full 60,000 training samples for up to 15 epochs with EarlyStopping(patience=3, restore_best_weights=True).
5. **Final Evaluation and Comparison**
   - Evaluate the tuned model on the same 10,000 test samples. Compare accuracy, per-digit metrics, and confusion matrix against the baseline.
   - Plot training accuracy and loss curves for both models to assess convergence behavior.



---

6. Performance Analysis

Architecture: Conv2D(32) → MaxPool → Conv2D(64) → MaxPool → Flatten → Dense(128) → Dense(10, Softmax)

Total Parameters: 225,034 | Optimizer: Adam(lr=0.001) | Epochs: 10 | Batch Size: 32

Dataset: MNIST | Train: 60,000 | Test: 10,000 | Classes: 10 digits (0–9)

Digit	Precision	Recall	F1-Score	Support
0	1.00	1.00	1.00	980
1	1.00	1.00	1.00	1135

2	0.99	0.99	0.99	1032
3	0.99	1.00	0.99	1010
4	0.99	0.99	0.99	982
5	0.98	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
Macro Avg	0.99	0.99	0.99	10000
Wtd. Avg	0.99	0.99	0.99	10000
Accuracy	—	—	99.17%	10000

1. **Near-Perfect Baseline Classification**

   The baseline CNN achieves 99.17% accuracy, correctly classifying 9,917 of 10,000 test images using fixed default hyperparameters and only 10 training epochs. This demonstrates the fundamental advantage of CNNs over traditional classifiers for image data: the convolutional layers automatically learn spatial feature detectors (strokes, curves, loops) that are directly relevant to digit recognition without any manual feature engineering.

2. **Digit 5 — Lowest Precision (0.98)**

   Digit 5 has the lowest precision (0.98) across all classes, as its upper horizontal stroke can be visually confused with a '6' and its curved bottom can resemble a '3'. This is consistent with established MNIST benchmarks where digit 5 is among the most frequently misclassified. Despite this, the F1-score remains 0.99.

3. **Digits 0 and 1 — Perfect Scores (1.00)**

   Digits 0 (full oval) and 1 (single vertical stroke) achieve precision and recall of 1.00, reflecting their highly distinctive visual structures. The convolutional filters can trivially differentiate these shapes from all other digits, resulting in zero misclassifications for both classes.

4. **Overall**

   With only 225,034 parameters and 10 epochs, the CNN already achieves near-human-level performance on MNIST. The two-block convolutional structure (32 → 64 filters) is sufficient to extract discriminative spatial features for all 10 digit classes without any regularisation or augmentation. This confirms that the CNN architecture is inherently well-matched to image classification tasks.

---

7. Hyperparameter Tuning

Before Tuning — Baseline Architecture

Hyperparameter	Baseline Value
filters_1 (Conv block 1)	32
filters_2 (Conv block 2)	64
dense_units	128
dropout	None
learning_rate	0.001
epochs	10
batch_size	32

Baseline Test Accuracy: 99.17%

Keras Tuner — RandomSearch Configuration:

- filters_1: Choice of [32, 64]
- filters_2: Choice of [64, 128]
- dense_units: Choice of [64, 128, 256]
- dropout: Float in [0.2, 0.5] (step 0.1)
- learning_rate: Choice of [1e–2, 1e–3, 1e–4]
- max_trials = 10, objective = 'val_accuracy', seed = 42
- Search subset: 10,000 samples | Search epochs: 5 per trial
- Final training: up to 15 epochs with EarlyStopping(patience=3, monitor='val_accuracy', restore_best_weights=True)

Best Hyperparameters Found:

Hyperparameter	Baseline Value	Best Value
filters_1 (Conv block 1)	32	64
filters_2 (Conv block 2)	64	128
dense_units	128	64
dropout	None	0.2
learning_rate	0.001	0.001
Epochs (stopped at)	10	12

After Tuning — Results:

Digit	Precision	Recall	F1-Score	Support
0	0.99	1.00	1.00	980

1	0.99	1.00	0.99	1135
2	1.00	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	1.00	0.99	0.99	982
5	0.99	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.98	0.99	0.99	1028
8	1.00	0.99	0.99	974
9	0.99	0.98	0.99	1009
Macro Avg	0.99	0.99	0.99	10000
Wtd. Avg	0.99	0.99	0.99	10000
Accuracy	—	—	99.25%	10000

Key Observations:

- Accuracy improved from 99.17% → 99.25% (+0.08 percentage points) — at this performance level, each additional correct classification represents a genuinely harder example; the total misclassifications reduced from 83 to 75
- Doubling both filter counts (32→64 in block 1, 64→128 in block 2) allows the network to learn a richer set of spatial feature detectors — more distinct edges, curves, and stroke junctions are captured per layer
- Reducing dense_units from 128 to 64 while adding Dropout(0.2) provides better regularisation: fewer fully connected weights reduce overfitting risk while dropout prevents neuron co-adaptation in the classifier head
- Learning rate remains 0.001 — the tuner confirmed the baseline was already at the optimal learning rate; neither 0.01 nor 0.0001 improved performance on the search subset
- Early stopping at epoch 12 (vs fixed 10 baseline) allows the tuned model two extra epochs of training, extracting more utility from the richer convolutional feature maps
- Digit 5 precision improved from 0.98 → 0.99 — the deeper filter bank in block 1 (64 filters) better distinguishes the upper horizontal stroke of '5' from visually similar digits
- Digit 7 now has the lowest precision (0.98) in the tuned model — the '7's diagonal stroke can be confused with '1' or '9' at certain handwriting styles, a known challenge even for deep networks

---

8. Code and Output – Without Tuning

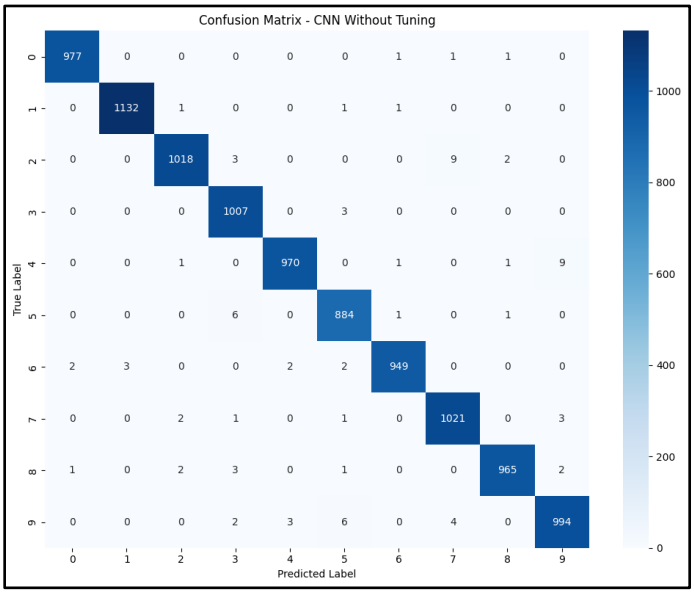
```
import numpy as np
import matplotlib.pyplot as plt
```

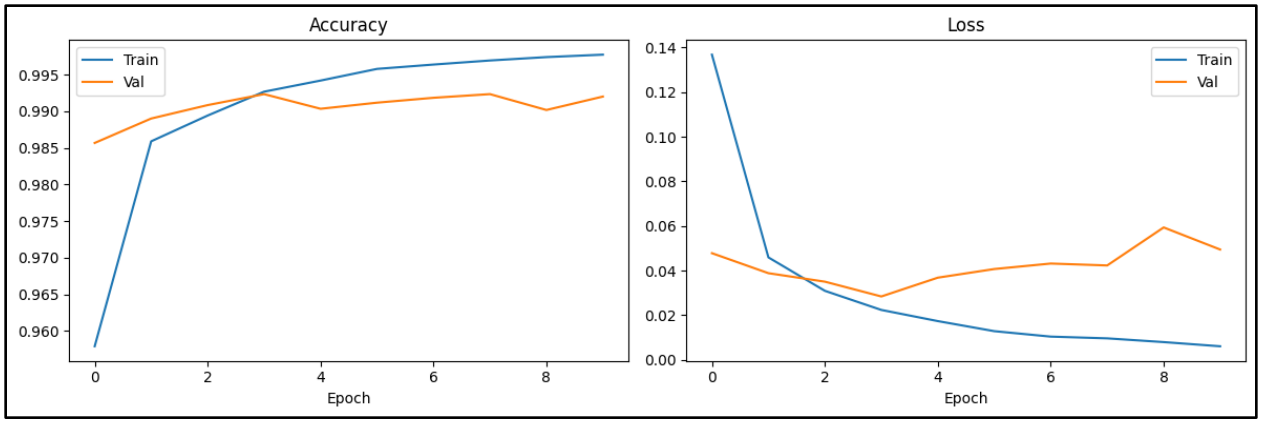
```
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import
classification_report, confusion_matrix
import seaborn as sns

# Load MNIST
(X_train, y_train), (X_test, y_test) =
keras.datasets.mnist.load_data()

# Preprocess
X_train = X_train.reshape(-1, 28, 28,
1).astype("float32") / 255.0
X_test  = X_test.reshape(-1, 28, 28,
1).astype("float32") / 255.0

print(f"Training samples:
{X_train.shape[0]}, Test samples:
{X_test.shape[0]}")
print(f"Image shape: {X_train.shape[1:]},
Classes: {np.unique(y_train)}")

# Build CNN
model = keras.Sequential([
    layers.Conv2D(32, (3, 3),
activation='relu', input_shape=(28, 28,
1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3),
activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer=keras.optimize
rs.Adam(learning_rate=0.001),
              loss='sparse_categorical_cross
entropy',
              metrics=['accuracy'])

model.summary()

# Train
history = model.fit(X_train, y_train,
epochs=10, batch_size=32,
                    validation_split=0.1,
verbose=1)

# Evaluate
test_loss, test_acc =
model.evaluate(X_test, y_test,
verbose=0)
print(f"\nTest Accuracy:
{test_acc*100:.2f}%")

y_pred =
np.argmax(model.predict(X_test),
axis=1)

print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=[str(i) for i in
range(10)]))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=range(10),
yticklabels=range(10))
plt.title('Confusion Matrix - CNN Without
Tuning')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()

# Training curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Train')
plt.plot(history.history['val_accuracy'],
label='Val')
plt.title('Accuracy'); plt.xlabel('Epoch');
plt.legend()

plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'],
label='Train')
plt.plot(history.history['val_loss'],
label='Val')
```

```
plt.title('Loss'); plt.xlabel('Epoch');
plt.legend()
plt.tight_layout()
plt.show()
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 64s 36ms/step - accuracy: 0.9579 - loss: 0.1368 - val_accuracy: 0.9857 - val_loss: 0.0478
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 58s 34ms/step - accuracy: 0.9859 - loss: 0.0459 - val_accuracy: 0.9890 - val_loss: 0.0389
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 57s 34ms/step - accuracy: 0.9894 - loss: 0.0309 - val_accuracy: 0.9908 - val_loss: 0.0351
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 57s 34ms/step - accuracy: 0.9927 - loss: 0.0224 - val_accuracy: 0.9923 - val_loss: 0.0284
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 82s 34ms/step - accuracy: 0.9942 - loss: 0.0174 - val_accuracy: 0.9903 - val_loss: 0.0368
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 56s 33ms/step - accuracy: 0.9958 - loss: 0.0129 - val_accuracy: 0.9912 - val_loss: 0.0407
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 56s 33ms/step - accuracy: 0.9964 - loss: 0.0104 - val_accuracy: 0.9918 - val_loss: 0.0432
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 56s 33ms/step - accuracy: 0.9969 - loss: 0.0096 - val_accuracy: 0.9923 - val_loss: 0.0423
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 84s 34ms/step - accuracy: 0.9974 - loss: 0.0080 - val_accuracy: 0.9902 - val_loss: 0.0594
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 82s 34ms/step - accuracy: 0.9977 - loss: 0.0061 - val_accuracy: 0.9920 - val_loss: 0.0495
```





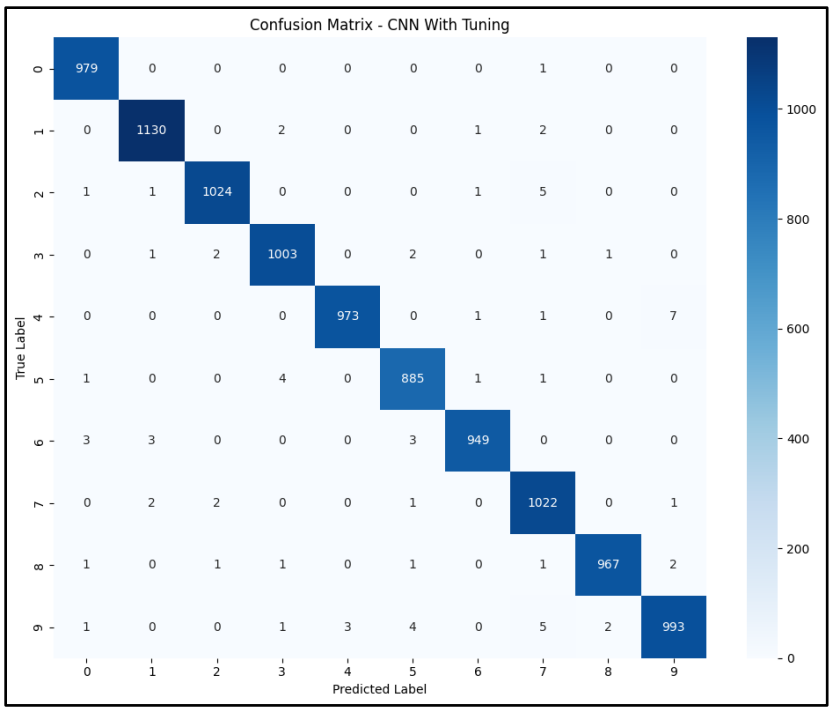
9. Code and Output – With Tuning

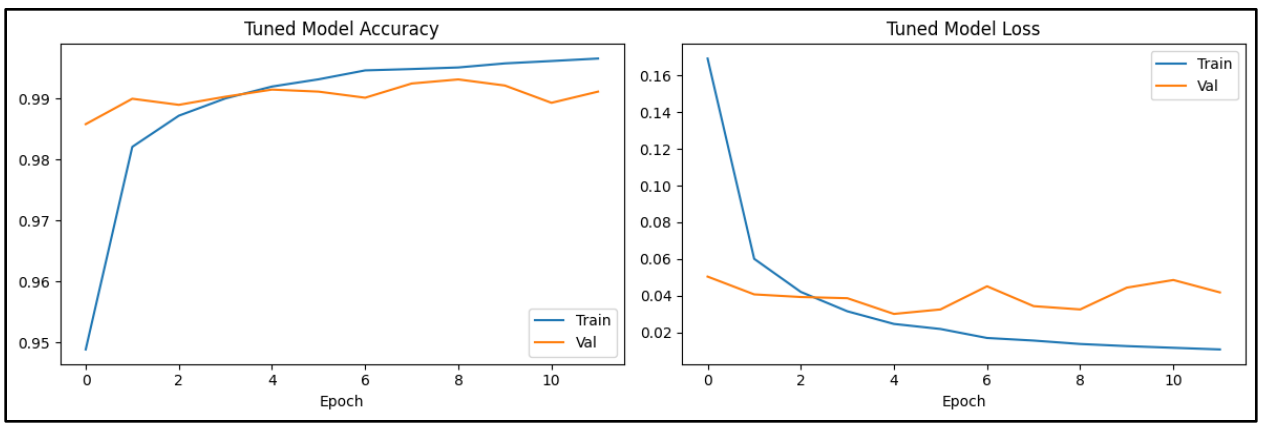
```
!pip install keras-tuner -q

import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt
from sklearn.metrics import
classification_report, confusion_matrix
import seaborn as sns

# Load and preprocess
(X_train, y_train), (X_test, y_test) =
keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 28, 28,
1).astype("float32") / 255.0
X_test  = X_test.reshape(-1, 28, 28,
1).astype("float32") / 255.0

# Subset for faster search
X_tune = X_train[:10000]
y_tune = y_train[:10000]

def build_model(hp):
    model = keras.Sequential()
    filters1 = hp.Choice('filters_1', [32,
64])
    model.add(layers.Conv2D(filters1, (3,
3), activation='relu', input_shape=(28,
28, 1)))
    model.add(layers.MaxPooling2D((2,
2)))

    filters2 = hp.Choice('filters_2', [64,
128])
    model.add(layers.Conv2D(filters2, (3,
3), activation='relu'))
    model.add(layers.MaxPooling2D((2,
2)))

    model.add(layers.Flatten())

    units = hp.Choice('dense_units', [64,
128, 256])
    model.add(layers.Dense(units,
activation='relu'))

    dropout = hp.Float('dropout', 0.2, 0.5,
step=0.1)
    model.add(layers.Dropout(dropout))

    model.add(layers.Dense(10,
activation='softmax'))

    lr = hp.Choice('learning_rate', [1e-2,
1e-3, 1e-4])
    model.compile(optimizer=keras.optimi
zers.Adam(learning_rate=lr),
                  loss='sparse_categorical_cro
ssentropy',
                  metrics=['accuracy'])
    return model

tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=10,
    seed=42,
    directory='cnn_tuning',
    project_name='mnist_cnn'
)

print("Starting hyperparameter search
(10 trials)...")
tuner.search(X_tune, y_tune, epochs=5,
validation_split=0.2, verbose=0)

best_hps =
tuner.get_best_hyperparameters(1)[0]
print("\nBest Hyperparameters:")
print(f"  filters_1     :
{best_hps.get('filters_1')}")
print(f"  filters_2     :
{best_hps.get('filters_2')}")
print(f"  dense_units   :
{best_hps.get('dense_units')}")
print(f"  dropout       :
{best_hps.get('dropout')}")
```

```
print(f"  learning_rate :
{best_hps.get('learning_rate')}")

# Train best model on full data
best_model =
tuner.hypermodel.build(best_hps)
early_stop =
keras.callbacks.EarlyStopping(monitor='
val_accuracy', patience=3,
                                restore_best
_weights=True)
history = best_model.fit(X_train, y_train,
epochs=15, batch_size=32,
                    validation_split=0.1,
callbacks=[early_stop], verbose=1)

print(f"\nStopped at epoch:
{len(history.history['accuracy'])}")

test_loss, test_acc =
best_model.evaluate(X_test, y_test,
verbose=0)
print(f"Test Accuracy:
{test_acc*100:.2f}%")

y_pred =
np.argmax(best_model.predict(X_test),
axis=1)

print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=[str(i) for i in
range(10)]))
```

```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
          xticklabels=range(10),
yticklabels=range(10))
plt.title('Confusion Matrix - CNN With
Tuning')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()

# Training curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Train')
plt.plot(history.history['val_accuracy'],
label='Val')
plt.title('Tuned Model Accuracy');
plt.xlabel('Epoch'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],
label='Train')
plt.plot(history.history['val_loss'],
label='Val')
plt.title('Tuned Model Loss');
plt.xlabel('Epoch'); plt.legend()
plt.tight_layout()
plt.show()
```

```
Epoch 1/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 121s 71ms/step - accuracy: 0.9489 - loss: 0.1693 - val_accuracy: 0.9858 - val_loss: 0.0504
Epoch 2/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 141s 71ms/step - accuracy: 0.9821 - loss: 0.0601 - val_accuracy: 0.9900 - val_loss: 0.0407
Epoch 3/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 122s 73ms/step - accuracy: 0.9872 - loss: 0.0421 - val_accuracy: 0.9890 - val_loss: 0.0393
Epoch 4/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 138s 70ms/step - accuracy: 0.9901 - loss: 0.0315 - val_accuracy: 0.9903 - val_loss: 0.0386
Epoch 5/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 119s 70ms/step - accuracy: 0.9920 - loss: 0.0246 - val_accuracy: 0.9915 - val_loss: 0.0300
Epoch 6/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 130s 77ms/step - accuracy: 0.9932 - loss: 0.0218 - val_accuracy: 0.9912 - val_loss: 0.0325
Epoch 7/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 128s 76ms/step - accuracy: 0.9946 - loss: 0.0169 - val_accuracy: 0.9902 - val_loss: 0.0451
Epoch 8/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 118s 70ms/step - accuracy: 0.9949 - loss: 0.0155 - val_accuracy: 0.9925 - val_loss: 0.0343
Epoch 9/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 144s 71ms/step - accuracy: 0.9951 - loss: 0.0136 - val_accuracy: 0.9932 - val_loss: 0.0325
Epoch 10/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 142s 70ms/step - accuracy: 0.9958 - loss: 0.0125 - val_accuracy: 0.9922 - val_loss: 0.0444
Epoch 11/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 119s 71ms/step - accuracy: 0.9962 - loss: 0.0116 - val_accuracy: 0.9893 - val_loss: 0.0485
Epoch 12/15
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 118s 70ms/step - accuracy: 0.9966 - loss: 0.0107 - val_accuracy: 0.9912 - val_loss: 0.0418

Stopped at epoch: 12
```





10. Conclusion

This experiment highlights the effectiveness of Convolutional Neural Networks for image classification using the MNIST dataset. A baseline CNN with two convolutional blocks (32 and 64 filters) and a Dense(128) layer achieves 99.17% accuracy in just 10 epochs, demonstrating how CNNs automatically learn spatial features like strokes and curves without manual feature engineering. Further optimization using Keras Tuner's RandomSearch yields a slightly improved model with deeper convolutional layers (64 and 128 filters), a smaller Dense(64) head, and Dropout(0.2), achieving 99.25% accuracy and reducing misclassifications from 83 to 75. Although the accuracy gain is modest, the key insight is architectural: allocating more capacity to convolutional layers while keeping the classifier compact and regularised leads to better performance, confirming that for clean image datasets, stronger feature extraction combined with controlled overfitting is more impactful than increasing dense layer complexity.